RYAN C. PASQUIN

JERSON B. CATAYLO

SECTION - C

NOVEMBER 11, 2026

Java OOP Laboratory Exercise: Campus Café Ordering System

If statement: Used the it as a command to add extra fee if the customer wants their pastry warmed or not, and their meals upsized. **If** they want their pastry to be warmed up, it adds extra fee, same goes **if** they want their meal upsized or not.

If-else statement: Used it as a command to display a certain word **if** the Boolean is **true**, **else** display its default state if its **false**. For example, I used it if the customer wants their pastry warmed (true) or not (false). if the Boolean is true, I told it to display **"(Warmed)"** beside their chosen pastry, else if its false, it will only display their pastry order.

Classes and Objects: I created 7 different classes to serve as blueprints for system and objects like customers, menu items, and orders inside **CafeApp.main**(). Basically, my other classes are the pillars of my main class (**CafeApp**).

Attributes and Behaviors: Defined private variables (like **item prices** and **customer names**) and methods across every class to bundle state and actions together.

Object Interaction: Connected **Order with Customer and MenuItem** objects so orders can **update customer loyalty points and calculate total receipts**.

Java Naming Conventions: Applied **PascalCase** for class names, **camelCase** for variables and methods, and **Mixed_Snake_Case** for fixed constants.

Default Constructor: Provided parameterless constructors for **MenuItem(), Beverage(), Pastry(), Meal(),** and **Customer()** to allow default object creation.

Parameterized Constructor: Created custom constructors across all **MenuItem subclasses**, **Customer**, and **Order** to initialize fields upon creation.

Constructor Overloading: Implemented **three different constructors** in the Customer class to allow creating customer objects with varying amounts of initial info.

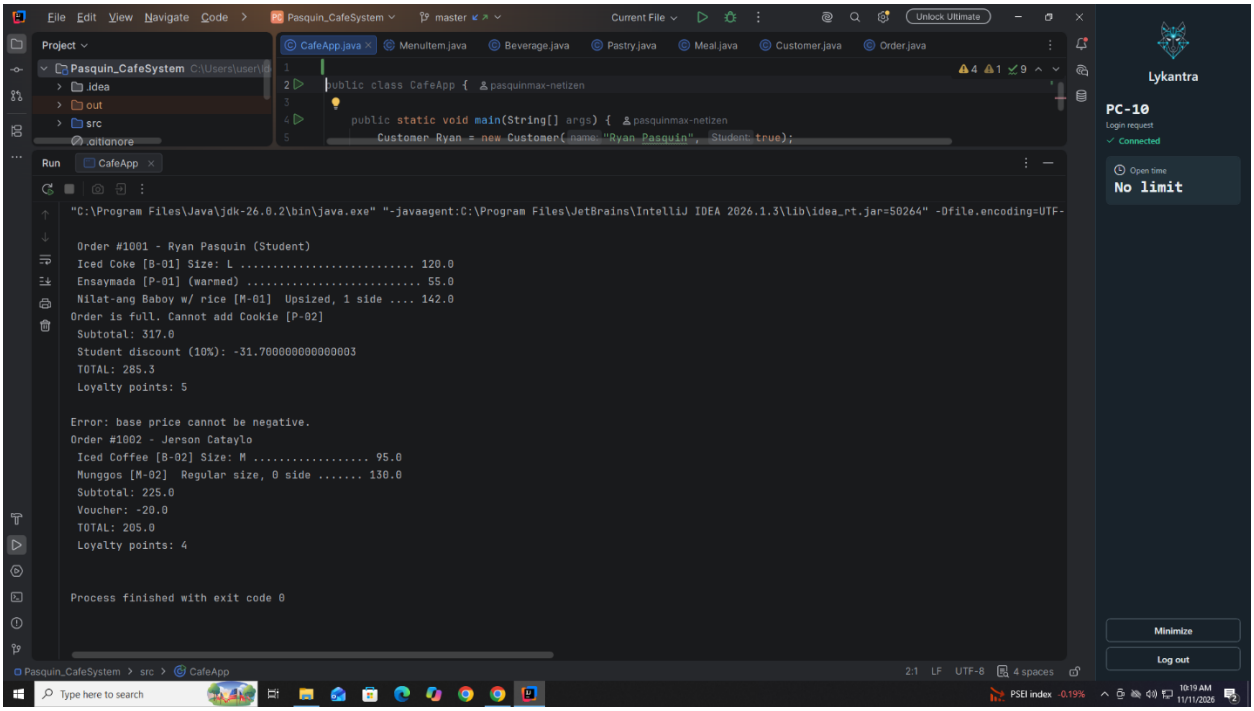
Public / Private / Protected: Used **public for accessible methods**, **private for hidden class fields**, and **protected** so subclasses can **access shared MenuItem fields** like name and basePrice.

Getters, Setters, Data Hiding: Encapsulated data using private variables with validated public getters and setters to block invalid inputs (like the **negative baseprice**).

Method Overloading: Created overloaded methods like Customer.addPoints(int) / addPoints(double) and Order.computeTotal() / computeTotal(double) to handle different parameter types.

Method Overriding: Overrode computePrice() and getDescription() in Beverage, Pastry, and Meal to give each item type its **custom price** and **description logic**.

Abstraction: Defined MenuItem as an abstract class with an abstract computePrice() method to enforce **custom price calculations in child classes**.



The biggest problem I encountered is the "Order is full. Cannot add [item]." The display was supposed to be displaying in the middle of the receipt text. Instead, it was always at the very top of the output which broke my final output placements. To fix this, the check was moved to execute directly at the top before processing the order receipt details, ensuring the capacity error message always appears at the center of the output where it should be.